

SEMINARIUM DYPLOMOWE INŻYNIERSKIE

- Część II

Rozwój portalu wspierającego obsługę rejsów dla biura Armatora
Wydział Oceanografii i Geografii UG

- Bartłomiej Krawisz 193319
- Paweł Pstrągowski 193473
- Stanisław Nieradko 193044

AGENDA

1. Harmonogram prac
2. Stan pracy
3. Przegląd zastosowanych technologii przy przepisywaniu frontendu
4. Porównanie narzędzi do testów automatycznych
5. Wybrane szczegóły implementacyjne w aplikacji

HARMONOGRAM PRAC

Termin	Zadanie
13 maja 2025	Zakończenie prac nad refaktorem aplikacji, wypuszczenie nowej wersji 2.0.0
czerwiec 2025 - październik 2025	Implementacja nowych funkcjonalności oraz naprawa znajduwanych błędów
21 września 2025	Wprowadzenie testów automatycznych do projektu
październik 2025 - grudzień 2025	Zapoznanie kolejnego zespołu z projektem
październik 2025 - grudzień 2025	Opis prac związanych z projektem w postaci pracy inżynierskiej
listopad 2025 - grudzień 2025	Wdrożenie nowej wersji aplikacji oraz ew. naprawa krytycznych błędów

Równolegle zaplanowane są regularne konsultacje z promotorem oraz klientami w interwałach odpowiednio 2 tygodni oraz miesiąca.

STAN PRACY

- Większość prac związanych z nowymi funkcjonalnościami dobiegła końca.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Aktualnie zajmujemy się:
 - Wdrażaniem nowej wersji na środowisko klienta
 - Wprowadzaniem nowego zespołu, który ma przejąć obowiązek utrzymywania aplikacji
 - Opisywaniem naszego projektu w pracy inżynierskiej.

- Nasz zespół zakończył już pracę nad dodawaniem nowych funkcjonalności.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Nowa wersja aplikacji została wdrożona na środowisko klienta 20 listopada 2025.
- Nowy zespół został wdrożony w projekt, pracują oni aktualnie nad zgłoszonymi problemami i kolejnymi funkcjonalnościami.
- Większość naszej pracy inżynierskiej została już napisana, pozostały tylko końcowe poprawki.

PRZEGLĄD ZASTOSOWANYCH TECHNOLOGII PRZY PRZEPISYWANIU FRONTENDU

PORÓWNIANIE NARZĘDZI DO TESTÓW AUTOMATYCZNYCH

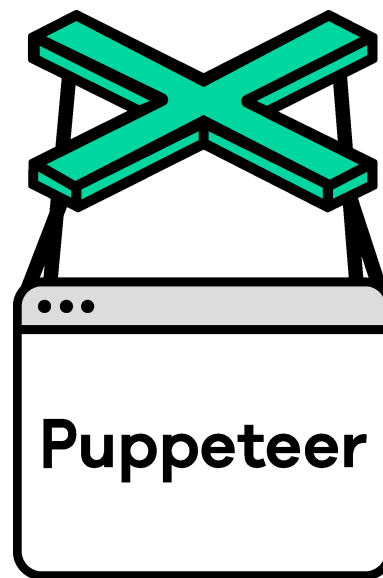
Najpopularniejsze rozwiązania stosowane w branży:



Selenium



Cypress



Puppeteer



Playwright

Selenium

Zalety:

- Szerokie wsparcie dla wielu przeglądarek, między innymi Google Chrome, Firefox, Safari, a nawet Internet Explorer

Wady:

- Brak oficjalnego wsparcia dla TypeScript
- Nieintuicyjne czekanie na oczekiwaną wartość

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

✗ Taki kod może w niektórych przypadkach nie zadziałać

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));

// Wait until checkbox becomes the wanted value
await driver.wait(async () => {
    const val = await textbox.getAttribute("value");
    return val === "Updated Value";
}, 5000);
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

Cypress

Zalety:

- Oficjalne wsparcie dla TypeScript
- Zaimplementowany domyślny, automatyczny sposób czekania na oczekiwaną wartość
- Udostępnia wiele narzędzi do inspekcji przebiegu testów

Wady:

- Duża część narzędzi jest dostępna tylko w dodatkowo płatnym Cypress Cloud
- W pełni oficjalnie wspierana jest tylko przeglądarka Google Chrome, z ograniczonym wsparciem dla przeglądarek Firefox i Safari

Cypress

```
// Click the button
```

```
cy.get('#myButton').click();
```

```
// Check updated textbox value
```

```
cy.get<HTMLInputElement>('#myTextbox').should('have.value', 'Updated Value');
```

✓ Taki kod zaczeka na oczekiwaną wartość przez ustalony z góry czas (np. domyślnie 30 s), nie trzeba samemu pisać kodu czekającego, jak to miało miejsce w Selenium

Puppeteer

Zalety:

- Oficjalne wsparcie dla TypeScript

Wady:

- Aktualnie kompatybilny tylko z Google Chrome i Firefox
- Podobnie jak w Selenium, nieintuicyjne czekanie na zmianę wartości

Puppeteer

```
// Click the button
await page.click("#myButton");

// Wait until textbox value becomes the expected value
await page.waitForFunction(
  () => (document.getElementById("myTextbox")
    as HTMLInputElement)?.value === "Updated Value",
  { timeout: 5000 }
);

// Get updated textbox value
const value = await page.$eval("#myTextbox",
  el => (el as HTMLInputElement).value);
assert.strictEqual(value, "Updated Value");
```

Playwright

Zalety:

- Kompatybilność ze wszystkimi współcześnie popularnymi przeglądarkami (Google Chrome, Firefox i Safari), jak i również z ich wersjami mobilnymi
- Oficjalne wsparcie dla TypeScript
- Domyślne czekanie na zmianę wartości na oczekiwaną
- Łatwa inspekcja przebiegu testów

Wady:

- Trochę gorsze narzędzia do inspekcji niż Cypress w płatnym wariancie

Playwright

```
// Click the button
await page.click('#myButton');

// Check updated textbox value
const textbox = page.locator('#myTextbox');
await expect(textbox).toHaveValue('Updated Value');
```

✓ Podobnie jak w Cypressie, tutaj również program automatycznie zaczeka przez ustalony czas

Podsumowanie

Po analizie różnych dostępnych rozwiązań, zdecydowaliśmy się na zastosowanie narzędzia **Playwright**.



WYBRANE SZCZEGÓŁY IMPLEMENTACYJNE W APLIKACJI

Dziękujemy za uwagę!